

Crime and Land Use: Exploring Violent Crime in London

Preparation

- [Github link](#)
- Number of words: 1,612
- Runtime: 0.008 hours (*Memory 8 GB, CPU Apple M3 chip*)
- Coding environment: SDS Docker
- License: this notebook is made available under the [Creative Commons Attribution license](#).
- Additional library [*libraries not included in SDS Docker or not used in this module*]:
 - No additional libraries needed.

```
In [1]: # Importing libraries

# For reading data
import requests
import zipfile
from io import StringIO

# For analysis
import numpy as np
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
from shapely.ops import nearest_points

# For clustering
from sklearn.preprocessing import RobustScaler
import sklearn.cluster as sklc
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn import metrics
from scipy.cluster.hierarchy import dendrogram

# For DiD
from statsmodels.formula.api import ols

# For plots
import matplotlib.pyplot as plt
import plotly
import plotly.express as px
import seaborn as sns
```

Table of contents

1. [Introduction & Background](#)

2. [Research questions](#)
3. [Data](#)
4. [Methodology](#)
5. [Analysis](#)
6. [Results and discussion](#)
7. [Conclusion](#)
8. [References](#)

1. Introduction & Background

[\[Back to top \]](#)

This study aims to explore violent crime in London at the LSOA level and how it relates to residential, industrial/commercial, and community-related land use. It first investigates the spatial distribution of clustering patterns emerging from crime and land use variables. It then conducts a case study of the 2018 opening of Coal Drops Yard (CDY) in King's Cross using a difference-in-differences approach to evaluate its impact on local violent crime.

1.1 Literature review

Crime and land use

Studying the impacts of residential and commercial activity on local crime, theorists like Jane Jacobs are optimistic about mixed land use, conferring the benefits of increasing "eyes on the street"-style social control and bolstering neighbourhood trust (Jacobs, 1961), factors that can reduce the risk of crime. Yet, others (e.g. Taylor, 1988) have associated residential/nonresidential mixing with a heightened sense of anonymity and decreased collective responsibility, increasing the propensity for crime. The latter position has contributed to general assumptions that residential land use exerts protective effects on crime, while commercial land use exacerbates it (Wo & Kim, 2020).

Yet, investigating the relationship between land use and crime in the context of Columbus, Ohio, Browning et al. (2010) found it nonlinear: while violent crime increased with commercial/residential density at low levels, it decreased when mixed land use increased beyond a threshold. Similarly, Wo and Kim's (2020) study of residential/nonresidential land use in San Francisco found that nonresidential land use and increased crime had a non-significant and inconclusive relationship. In other words, nonresidential land use cannot always be associated with higher crime in the area (Stucky & Ottensmann, 2009), demonstrating the limitations of a binary understanding of residential/nonresidential land use and crime.

Violent crime in London

This study adopts a specific interest in violent crimes. As a type of incident that necessarily involves individuals, the consequences of violent crimes extend beyond the incident; sentiments of fear and anxiety surrounding violent crimes negatively affect perceptions of neighbourhoods, in turn diminishing their vitality, commercial success, and residents' quality of life (Nelson et al., 2001). For their far-reaching social, psychological, and economic ramifications, violent crimes are a crucial subset of overall crime. This is of further importance in the UK, which has seen significant upward trends in youth and weapon-related crime since 2012/2013 (Haylock et al., 2020) and worrying upticks in weapon-related deaths/homicides.

Coal Drops Yard

The redevelopment of King's Cross is a timely and relevant case for analysis: over the 2010s, major renovation works transformed the derelict area from one in decline to a bustling public space and retail/commercial complex, with CDY opening in October 2018 and hosting a wide range of shops, restaurants, and other businesses. These changes in land use characteristics lend themselves to further study in relation to local violent crime.

2. Research questions

[\[Back to top \]](#)

This study aims to (i) uncover patterns at the intersection of violent crime and land use in London's context and (ii) determine if conclusions can be drawn regarding CDY's impact on local violent crime. As such, my research questions are as follows:

RQ1: What clusters and distributions can be observed from violent crime rates and land use characteristics across LSOAs?

RQ2: Did the Coal Drops Yard development in King's Cross have causal impacts on local violent crime?

3. Data

[\[Back to top \]](#)

3.1 Crime data

The key dataset this study uses is crime data on number of crimes according to major/minor crime category for each LSOA. Crime data is available in two time series: the most recent 24 months (April 2023 - March 2025) and all historical months with data (April 2010 - January 2023).

3.2 Land use data

Governmental land use data provides information on land use patterns across England according to subcategories of developed (e.g. residential) and non-developed land use (e.g. agricultural) land use.

3.3 Geographical boundary data

To incorporate geographical analysis, GIS boundary files for London LSOAs are used. These also contain basic demographic information.

Summary of data used:

Dataset	Description	Source	Timeframe(s)
Violent crime data	Counts of cases by category	London MPS	201004-202301, 202304-202503
Land use data	Proportion land occupied by category	UK Ministry of Housing, Communities & Local Government	2022
Geographical boundary data	LSOA boundaries and demographics	Greater London Authority (GLA)	2011

4. Methodology

[\[Back to top \]](#)

4.1 Data preparation

- **Crime data** was filtered to relevant categories and aggregated across categories for each time period;
- **Land use data** was filtered to categories of interest (residential, industrial/commercial, and community service);
- **Boundary data** was filtered to London LSOAs.

The datasets were merged to combine the above information. Crime rates were calculated using demographic data to account for population size. To avoid NA values influencing the interpretation of results, some LSOAs were dropped in the analysis stage. After ensuring that the remaining data was correctly merged and had the data (of correct type) required, it then underwent analysis.

4.2 Methods

4.2.1 Cluster analysis

Clustering is an unsupervised machine learning approach that works with unlabelled data, dividing data points into groups based on attribute similarity. The RobustScaler is used here to scale data as it is robust to outliers and is suited to the highly skewed crime data (as shown below). It eliminates the median value and scales data according to the interquartile range (between 25th and 75th quantile) (Thakker and H. Buch, 2024). This study uses k-Means and agglomerative clustering methods on historical crime data

k-Means clustering divides a dataset into groups based on the proximity of points in a multidimensional space. Points are assigned to the nearest randomly-placed centroid; centroids are subsequently recalculated as the new cluster mean until assignments are stable (Vanderplas, 2022)

This study uses the *k*-Means function from `sklearn` and the algorithm is run 10 times (default `n_init` value) with different centroid seeds, producing a final result with the best sum of squared errors (SSE) output.

While *k*-Means is suitable for the numerical data in this study, it is sensitive to outliers, which can introduce inaccuracies in clusters; it also requires knowledge of the number of clusters (*k*).

Agglomerative hierarchical clustering is a bottom-up method that gradually merges data points into larger clusters based on the proximity of instances. Centroids of new clusters are computed; thresholds are expanded until all points are combined in one cluster.

In contrast to *k*-means clustering, agglomerative clustering does not require predefined parameters, making it better suited to real-world data used in this study, where the number of clusters is unknown (Bouguettaya et al., 2015).

4.2.2 Difference-in-differences (DiD)

For the case study, a DiD approach is used to analyse if a cause-and-effect relationship can be inferred between CDY and local crime rates. Taking its opening in October 2018 as the treatment, 2011-2022 crime data is used here as it allows for pre- and post-treatment data analysis. The key components of the DiD analysis were undertaken as follows:

- **Identifying treated group:** As crime data for the CDY's LSOA (Camden 022B) was missing, the closest LSOA for which crime data was available was identified. While perhaps not ideal, investigating effects on a nearby LSOA as a proxy is nevertheless meaningful as it provides insight crime spillover across neighbourhoods.
- **Identifying control group:** Pearson correlation coefficients for pre-treatment crime rates between the treated and non-treated LSOAs was calculated. The LSOA with the highest correlation was selected as the treatment group.
- **Assessing treatment effect:** An OLS model assessed the statistical significance of the change in violent crime as a result of CDY's opening.

DiD estimates the overall effect of the CDY opening by the following equation:

$$Y_{it} = \beta_0 + \beta_1 \text{Post}_t + \beta_2 \text{Treat}_i + \beta_3 (\text{Treat}_i \times \text{Post}_t) + \epsilon_{it}$$

with coefficients as follows:

	Control	Treatment
Before	β_0	$\beta_0 + \beta_2$
After	$\beta_0 + \beta_1$	$\beta_0 + \beta_1 + \beta_2 + \beta_3$

4.3 Analysis workflow



5. Analysis

This section contains the code used to run the analysis as per the workflow outlined in the preceding section, accompanied by explanatory comments.

```
In [2]: # Function for reading in data
def load_csv_from_url(url, **kwargs):
    response = requests.get(url)
    if response.status_code == 200:
        data = StringIO(response.text)
        df = pd.read_csv(data, **kwargs)
        print("CSV data loaded successfully.")
        return df
    else:
        print(f"Failed to fetch the file: {response.status_code}")
```

5.1 Reading in data

Crime data (202304 - 202503)

```
In [3]: # Reading in data
url = "https://raw.githubusercontent.com/jyjylow/casa0006-assessment/refs/heads/main/crime_data.csv"
crimedf = load_csv_from_url(url)

# Peeking at crime data
crimedf.head()
```

CSV data loaded successfully.

Out [3]:

	LSOA Code	LSOA Name	Borough	Major Category	Minor Category	202304	202305	20
0	E01000006	Barking and Dagenham 016A	E09000002	ARSON AND CRIMINAL DAMAGE	CRIMINAL DAMAGE	1	0	
1	E01000006	Barking and Dagenham 016A	E09000002	BURGLARY	BURGLARY - RESIDENTIAL	0	0	
2	E01000006	Barking and Dagenham 016A	E09000002	BURGLARY	BURGLARY IN A DWELLING	1	0	
3	E01000006	Barking and Dagenham 016A	E09000002	DRUG OFFENCES	POSSESSION OF DRUGS	0	0	
4	E01000006	Barking and Dagenham 016A	E09000002	DRUG OFFENCES	TRAFFICKING OF DRUGS	1	0	

5 rows × 29 columns

In [4]:

```
# Checking column names and dtype  
crimedf.dtypes
```

```
Out[4]: LSOA Code      object
        LSOA Name    object
        Borough      object
        Major Category object
        Minor Category object
        202304        int64
        202305        int64
        202306        int64
        202307        int64
        202308        int64
        202309        int64
        202310        int64
        202311        int64
        202312        int64
        202401        int64
        202402        int64
        202403        int64
        202404        int64
        202405        int64
        202406        int64
        202407        int64
        202408        int64
        202409        int64
        202410        int64
        202411        int64
        202412        int64
        202501        int64
        202502        int64
        202503        int64
        dtype: object
```

```
In [5]: # Understanding the categories of crime reports
print(f"Counts of reports by major crime category:")
print(crimedf["Major Category"].value_counts().reset_index())
```

```
Counts of reports by major crime category:
   Major Category  count
0             THEFT  16467
1  VEHICLE OFFENCES  15613
2           BURGLARY  13999
3  PUBLIC ORDER OFFENCES  13194
4  VIOLENCE AGAINST THE PERSON  10152
5           DRUG OFFENCES   9110
6  ARSON AND CRIMINAL DAMAGE   7278
7             ROBBERY   7000
8  MISCELLANEOUS CRIMES AGAINST SOCIETY  4527
9  POSSESSION OF WEAPONS   3528
```

```
In [6]: # "VIOLENCE AGAINST THE PERSON" reports are evidently relevant to the study.
# But other categories might also be important, so I'm taking a look at the
# Displaying main and subcategories of crime reports
crime_categories = crimedf[["Major Category", "Minor Category"]].drop_duplicates()
crime_categories
```

Out [6] :

	Major Category	Minor Category
0	ARSON AND CRIMINAL DAMAGE	ARSON
1	ARSON AND CRIMINAL DAMAGE	CRIMINAL DAMAGE
2	BURGLARY	BURGLARY - RESIDENTIAL
3	BURGLARY	BURGLARY BUSINESS AND COMMUNITY
4	BURGLARY	BURGLARY IN A DWELLING
5	BURGLARY	BURGLARY NON-DWELLING
6	DRUG OFFENCES	POSSESSION OF DRUGS
7	DRUG OFFENCES	TRAFFICKING OF DRUGS
8	MISCELLANEOUS CRIMES AGAINST SOCIETY	MISC CRIMES AGAINST SOCIETY
9	POSSESSION OF WEAPONS	POSSESSION OF WEAPONS
10	PUBLIC ORDER OFFENCES	OTHER OFFENCES PUBLIC ORDER
11	PUBLIC ORDER OFFENCES	PUBLIC FEAR ALARM OR DISTRESS
12	PUBLIC ORDER OFFENCES	RACE OR RELIGIOUS AGG PUBLIC FEAR
13	PUBLIC ORDER OFFENCES	VIOLENT DISORDER
14	ROBBERY	ROBBERY OF BUSINESS PROPERTY
15	ROBBERY	ROBBERY OF PERSONAL PROPERTY
16	THEFT	BICYCLE THEFT
17	THEFT	OTHER THEFT
18	THEFT	SHOPLIFTING
19	THEFT	THEFT FROM THE PERSON
20	VEHICLE OFFENCES	AGGRAVATED VEHICLE TAKING
21	VEHICLE OFFENCES	INTERFERING WITH A MOTOR VEHICLE
22	VEHICLE OFFENCES	THEFT FROM A VEHICLE
23	VEHICLE OFFENCES	THEFT OR UNAUTH TAKING OF A MOTOR VEH
24	VIOLENCE AGAINST THE PERSON	HOMICIDE
25	VIOLENCE AGAINST THE PERSON	VIOLENCE WITH INJURY
26	VIOLENCE AGAINST THE PERSON	VIOLENCE WITHOUT INJURY

Based on this, I identified that:

- "PUBLIC ORDER OFFENCES: VIOLENT DISORDER"
- "VEHICLE OFFENCES: AGGRAVATED VEHICLE TAKING"

are also highly likely to be relevant to my study as they entail a degree of person-to-person physical violence. These will thus be included in analysis.

```
In [7]: # Filtering crimedf to relevant rows/cols
crimedf = crimedf[
    (crimedf["Major Category"] == "VIOLENCE AGAINST THE PERSON") |
    ((crimedf["Major Category"] == "PUBLIC ORDER OFFENCES") & (crimedf["Minor Category"] == "VIOLENCE AGAINST THE PERSON")) |
    ((crimedf["Major Category"] == "VEHICLE OFFENCES") & (crimedf["Minor Category"] == "VEHICLE OFFENCES"))
]
```

Crime data (Historical)

```
In [8]: # Reading in data
url = "https://raw.githubusercontent.com/jylylow/casa0006-assessment/refs/heads/main/crimehistdf.csv"
crimehistdf = load_csv_from_url(url)

# Peeking at crime data
crimehistdf.head()
```

CSV data loaded successfully.

```
Out[8]:
```

	LSOA Code	LSOA Name	Borough	Major Category	Minor Category	201903	201904	201905
0	E01000006	Barking and Dagenham 016A	E09000002	ARSON AND CRIMINAL DAMAGE	ARSON	1	0	0
1	E01000006	Barking and Dagenham 016A	E09000002	ARSON AND CRIMINAL DAMAGE	CRIMINAL DAMAGE	1	2	0
2	E01000006	Barking and Dagenham 016A	E09000002	BURGLARY	BURGLARY BUSINESS AND COMMUNITY	0	0	0
3	E01000006	Barking and Dagenham 016A	E09000002	BURGLARY	BURGLARY IN A DWELLING	1	0	0
4	E01000006	Barking and Dagenham 016A	E09000002	DRUG OFFENCES	POSSESSION OF DRUGS	2	2	0

5 rows × 159 columns

```
In [9]: # Checking column names and dtype
crimehistdf.dtypes
```

```
Out[9]: LSOA Code      object
LSOA Name      object
Borough        object
Major Category  object
Minor Category  object
...
202209         int64
202210         int64
202211         int64
202212         int64
202301         int64
Length: 159, dtype: object
```

```
In [10]: # Taking a look at range of available data
yyyyymm_list = sorted([col for col in crimehistdf.columns if col.isdigit()])
print(f"Earliest month available: {yyyyymm_list[0]}")
print(f"Latest month available: {yyyyymm_list[-1]}")
```

Earliest month available: 201004
Latest month available: 202301

```
In [11]: # Similarly, filtering crimehistdf to relevant rows/cols based on violent cr

# Filtering dataset to crimes of "VIOLENCE AGAINST THE PERSON" and other rel
crimehistdf = crimehistdf[
    (crimehistdf["Major Category"] == "VIOLENCE AGAINST THE PERSON") |
    ((crimehistdf["Major Category"] == "PUBLIC ORDER OFFENCES") & (crimehist
    ((crimehistdf["Major Category"] == "VEHICLE OFFENCES") & (crimehistdf["M
```

Land use data

```
In [12]: # Reading in data
# NOTE: this dataset includes all of England's LSOAs.
# NOTE: the column names on the CSV have been edited manually before uploadi
url = "https://raw.githubusercontent.com/jylylow/casa0006-assessment/refs/he
landuse = load_csv_from_url(url, skiprows = 5, low_memory = False)
landuse.head()
```

CSV data loaded successfully.

Out[12]:

	LSOA code	LSOA name	MSOA code	MSOA name	House of Commons Library MSOA Names	Community buildings	Leisure and recreational buildings	Commun serv To
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
1	E92000001	England	NaN	NaN	NaN	0.7	0	
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
3	E12000004	East Midlands	NaN	NaN	NaN	0.6	0	
4	E12000006	East of England	NaN	NaN	NaN	0.7	0	

5 rows × 49 columns

```
In [13]: # Filtering to relevant land use categories
landuse = landuse[["LSOA code", "LSOA name", "Residential Total", "Industry

# Checking column names and types
landuse.dtypes
```

```
Out[13]: LSOA code          object
LSOA name          object
Residential Total    float64
Industry and commerce Total    object
Community service Total    object
dtype: object
```

```
In [14]: # The columns in landuse represent the proportion of LSOA area occupied by t
# I'll rename the columns to reflect this more intuitively.
landuse.rename(columns={'Residential Total': 'res_prop',
                        'Industry and commerce Total': 'indcom_prop',
                        'Community service Total': 'commun_prop'},
               inplace=True)

# Converting industry/commerce and community service land proportions to float
cols_to_float = ["res_prop", "indcom_prop", "commun_prop"]
for col in cols_to_float:
    landuse[col] = pd.to_numeric(landuse[col], errors="coerce")

# Checking that column names and types have been correctly changed
landuse.dtypes
```

```
Out[14]: LSOA code      object
LSOA name      object
res_prop      float64
indcom_prop    float64
commun_prop    float64
dtype: object
```

London LSOA boundaries and demographic data

```
In [15]: # Reading in London LSOA boundaries
lsoa_url = "https://github.com/jylylow/casa0006-assessment/raw/main/data/statistical-gis-boundaries-london.zip"
lsoa_response = requests.get(lsoa_url)
with open("statistical-gis-boundaries-london.zip", "wb") as f:
    f.write(lsoa_response.content)

# Extracting contents of zipped folder to 'data' folder (within project directory)
with zipfile.ZipFile("statistical-gis-boundaries-london.zip", "r") as zip_ref:
    zip_ref.extractall("data/lsoa_shapefiles")
```

```
In [16]: # Creating geodataframe from London LSOA shapefile
gdf = gpd.read_file("data/lsoa_shapefiles/statistical-gis-boundaries-london.zip")
gdf.head()
```

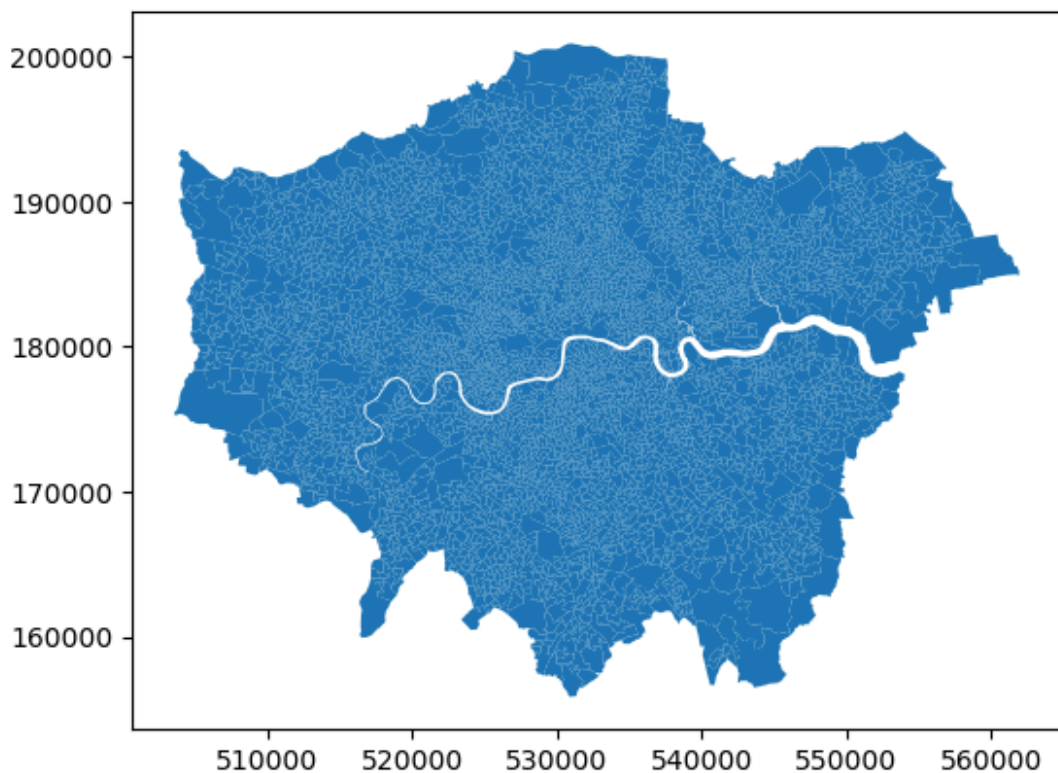
Out [16]:

	LSOA11CD	LSOA11NM	MSOA11CD	MSOA11NM	LAD11CD	LAD11NM	RGN11CD
--	----------	----------	----------	----------	---------	---------	---------

0	E01000001	City of London 001A	E02000001	City of London 001	E09000001	City of London	E12000007
1	E01000002	City of London 001B	E02000001	City of London 001	E09000001	City of London	E12000007
2	E01000003	City of London 001C	E02000001	City of London 001	E09000001	City of London	E12000007
3	E01000005	City of London 001E	E02000001	City of London 001	E09000001	City of London	E12000007
4	E01000006	Barking and Dagenham 016A	E02000017	Barking and Dagenham 016	E09000002	Barking and Dagenham	E12000007

In [17]: `# Peeking at gdf`
`gdf.plot()`

Out [17]: `<Axes: >`



```
In [18]: # Inspecting gdf info and CRS
gdf.info()
print(f"\nCRS: {gdf.crs}")
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 4835 entries, 0 to 4834
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   LSOA11CD    4835 non-null   object
1   LSOA11NM    4835 non-null   object
2   MSOA11CD    4835 non-null   object
3   MSOA11NM    4835 non-null   object
4   LAD11CD     4835 non-null   object
5   LAD11NM     4835 non-null   object
6   RGN11CD     4835 non-null   object
7   RGN11NM     4835 non-null   object
8   USUALRES    4835 non-null   int32
9   HHOLDRES    4835 non-null   int32
10  COMESTRES   4835 non-null   int32
11  POPDEN      4835 non-null   float64
12  HHOLDS      4835 non-null   int32
13  AVHHOLDSZ   4835 non-null   float64
14  geometry    4835 non-null   geometry
dtypes: float64(2), geometry(1), int32(4), object(8)
memory usage: 491.2+ KB
```

```
CRS: PROJCS["OSGB36 / British National Grid",GEOGCS["OSGB36",DATUM["Ordnance
_Survey_of_Great_Britain_1936",SPHEROID["Airy 1830",6377563.396,299.3249646,
AUTHORITY["EPSG","7001"]],AUTHORITY["EPSG","6277"]],PRIMEM["Greenwich",0],UN
IT["Degree",0.0174532925199433]],PROJECTION["Transverse_Mercator"],PARAMETER
["latitude_of_origin",49],PARAMETER["central_meridian",-2],PARAMETER["scale_
factor",0.999601272],PARAMETER["false_easting",400000],PARAMETER["false_nor
thing",-100000],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AXIS["Easting",EAS
T],AXIS["Northing",NORTH]].
```

5.2 Data transformation

[\[Back to top \]](#)

```
In [19]: # Summing YYYYMM columns so that we have the aggregate sum of reports for pa

# 1. Identifying YYYYMM columns
monthly_cols = [col for col in crimedf.columns if col.isdigit()]

# 2. Summing up YYYYMM columns
crime_past24_agg = pd.DataFrame() # Creating empty dataframe for aggregates
crime_past24_agg["total"] = crimedf.groupby(['LSOA Code',
                                             'LSOA Name']) [monthly_cols].sum

# 3. Resetting index
crime_past24_agg = crime_past24_agg.reset_index()
print("Total Violent Crime Counts (Past 24 Months):")
crime_past24_agg.head()
```

Total Violent Crime Counts (Past 24 Months):

Out [19]:

	LSOA Code	LSOA Name	total
0	E01000006	Barking and Dagenham 016A	67
1	E01000007	Barking and Dagenham 015A	363
2	E01000008	Barking and Dagenham 015B	122
3	E01000009	Barking and Dagenham 016B	193
4	E01000011	Barking and Dagenham 016C	122

In [20]:

```
# Merging crime_past24 with LSOA shp

merged_crime24 = pd.merge(gdf, crime_past24_agg,
                          left_on='LSOA11CD', right_on='LSOA Code',
                          how='inner')
merged_crime24 = merged_crime24[["LSOA11CD", "LSOA11NM",
                                "USUALRES", "geometry", "total"]]
merged_crime24.head()
```

Out [20]:

	LSOA11CD	LSOA11NM	USUALRES	geometry	total
0	E01000006	Barking and Dagenham 016A	1703	POLYGON ((545122.049 184314.931, 545271.917 18...	67
1	E01000007	Barking and Dagenham 015A	1391	POLYGON ((544180.347 184700.582, 544317.196 18...	363
2	E01000008	Barking and Dagenham 015B	1544	MULTIPOLYGON (((543741.001 184566, 543806.15 1...	122
3	E01000009	Barking and Dagenham 016B	1773	POLYGON ((544499.837 184704.366, 544630.292 18...	193
4	E01000011	Barking and Dagenham 016C	1634	POLYGON ((544523.402 184789.039, 544667.055 18...	122

In [21]:

```
# Comparing shapes of gdfs pre- and post-merge to check if any LSOAs were not matched
print(f"No. of LSOAs in original gdf: {gdf.shape[0]}")
print(f"No. of LSOAs in merged gdf: {merged_crime24.shape[0]}")
print(f"{gdf.shape[0] - merged_crime24.shape[0]} LSOAs were not matched.")
```

No. of LSOAs in original gdf: 4835

No. of LSOAs in merged gdf: 4653

182 LSOAs were not matched.

In [22]:

```
# It looks like there are some LSOAs that appear in the crime reports data but not in shp
# ...leading to less-than-perfect matching of LSOA codes between the two.
```

```
# Identifying which LSOAs are in crime data but not in shp
not_in_gdf = set(crime_past24_agg['LSOA Code']) - set(gdf['LSOA11CD'])
no_match = crime_past24_agg[crime_past24_agg['LSOA Code'].isin(not_in_gdf)]
print(f"LSOAs in crime df but not in shp: {len(no_match['LSOA Code']).unique()}")

# Identifying which LSOAs are in shp but not in crime data
# This will explain the gaps in the following visualisations of merged data.
not_in_crimedf = set(gdf['LSOA11CD']) - set(crime_past24_agg['LSOA Code'])
no_match_crime = gdf[gdf['LSOA11CD'].isin(not_in_crimedf)]
print(f"LSOAs in shp but not in crime data: {len(no_match_crime['LSOA11CD']).unique()}")
```

LSOAs in crime df but not in shp: 335

LSOAs in shp but not in crime data: 182

```
In [23]: # Calculating yearly average crime rates by:
# 1: dividing by 2 to get 12-monthly average
# 2: dividing by 1000 residents to get crime rate per 1000 residents
merged_crime24.loc[:, 'viol_rate'] = ((merged_crime24.loc[:, 'total']/2)/merged_crime24.loc[:, 'USUALRES'])

# Understanding the range of crime rates
print(f"Based on data from April 2024 to March 2025, {merged_crime24.loc[merged_crime24['viol_rate'].min():merged_crime24['viol_rate'].max(), 'LSOA11CD'].values[0]} had the lowest yearly violent crime rate of {merged_crime24.loc[merged_crime24['viol_rate'].min(), 'viol_rate'].values[0]} cases per 1000 residents) and {merged_crime24.loc[merged_crime24['viol_rate'].max():merged_crime24['viol_rate'].min(), 'LSOA11CD'].values[0]} had the highest yearly violent crime rate of {merged_crime24.loc[merged_crime24['viol_rate'].max(), 'viol_rate'].values[0]} cases per 1000 residents.
```

Based on data from April 2024 to March 2025, Harrow 004D had the lowest yearly violent crime rate of 1.10 cases per 1000 residents) and Westminster 018A had the highest yearly violent crime rate of 632.35 cases per 1000 residents.

```
In [24]: # Merging land use data

merged_crime24 = pd.merge(merged_crime24, landuse,
                           left_on='LSOA11CD', right_on='LSOA code',
                           how='inner')
merged_crime24 = merged_crime24[["LSOA11CD", "LSOA11NM", "USUALRES", # demographic
                                "res_prop", "indcom_prop", "commun_prop", # crime rates
                                "total", "viol_rate", # geometry]]

merged_crime24.head()
```

```
Out[24]:
```

	LSOA11CD	LSOA11NM	USUALRES	res_prop	indcom_prop	commun_prop	total
0	E01000006	Barking and Dagenham 016A	1703	21.0	NaN	0.1	67
1	E01000007	Barking and Dagenham 015A	1391	13.5	3.5	9.2	363
2	E01000008	Barking and Dagenham 015B	1544	5.6	9.6	1.8	122
3	E01000009	Barking and Dagenham 016B	1773	22.7	5.5	0.7	193
4	E01000011	Barking and Dagenham 016C	1634	29.2	0.7	0.0	122

```
In [25]: # Checking for NaN values
merged_crime24.isna().sum()
```

```
Out[25]: LSOA11CD          0
LSOA11NM          0
USUALRES          0
res_prop          0
indcom_prop      553
commun_prop      294
total            0
viol_rate        0
geometry          0
dtype: int64
```

```
In [26]: # To avoid NA values affecting the outcomes of analysis, I've chosen to drop
merged_crime24_drop = merged_crime24.dropna()

# Checking
print(f"{merged_crime24.shape[0] - merged_crime24_drop.shape[0]} LSOAs dropped")
```

772 LSOAs dropped after removing rows with NA values.

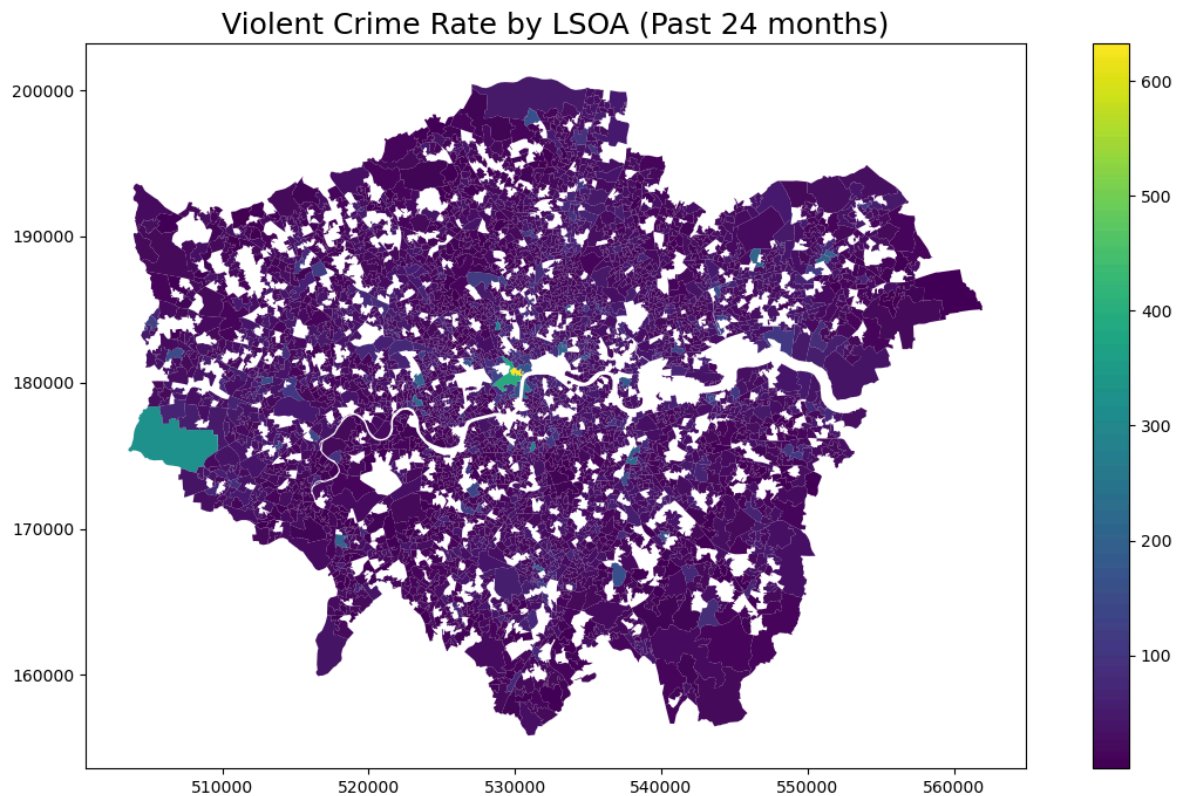
5.3 Summary statistics and exploratory plots

```
In [27]: # Understanding data distribution of violent crime rates and land use for the
crime24_summary = merged_crime24_drop[['USUALRES', 'res_prop', 'indcom_prop', 'commun_prop', 'viol_rate']]
crime24_summary = crime24_summary.T
crime24_summary
```

```
Out[27]:
```

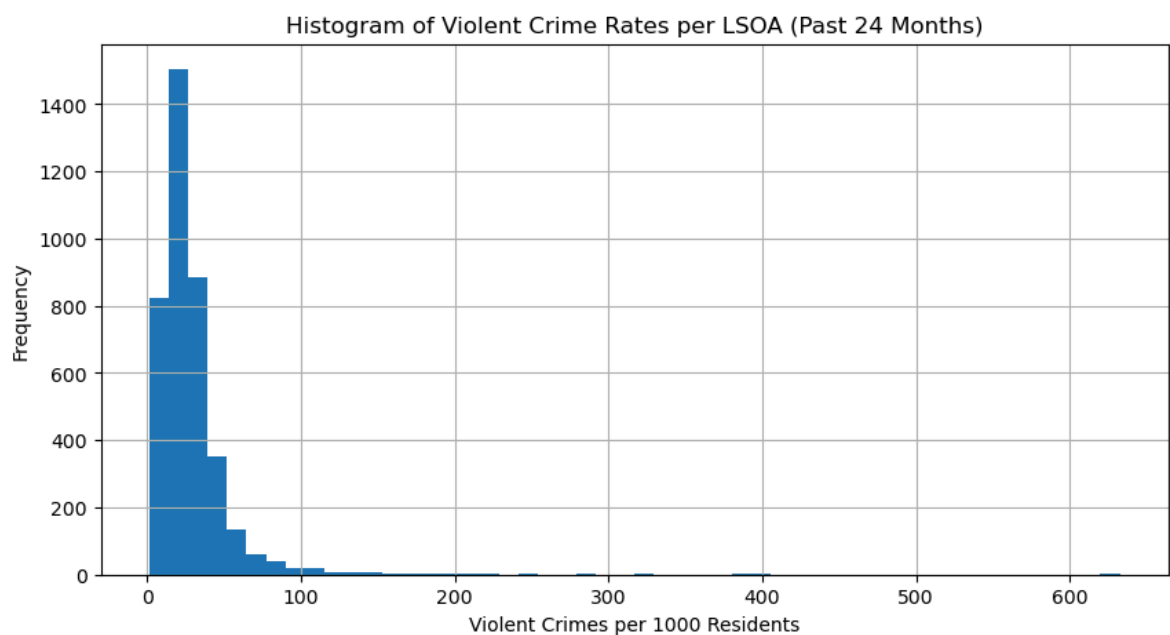
	count	mean	std	min	25%	50%	75%	max
USUALRES	3881.0	1695.18	240.77	990.00	1539.00	1665.00	1825.00	3314.00
res_prop	3881.0	16.92	7.57	0.40	11.80	16.50	21.70	45.30
indcom_prop	3881.0	2.35	3.83	0.00	0.20	0.90	2.60	33.40
commun_prop	3881.0	5.52	7.78	0.00	0.70	2.80	7.10	80.50
viol_rate	3881.0	28.75	26.22	1.64	15.48	23.69	33.78	632.35

```
In [28]: # Quickly visualising crime reports across LSOAs
merged_crime24_drop.plot(column='viol_rate',
                           legend=True,
                           cmap='viridis',
                           figsize=(12, 7))
plt.title("Violent Crime Rate by LSOA (Past 24 months)", fontsize=18)
plt.tight_layout()
plt.show()
```



In [29]: *# The colouring in the preliminary plot above isn't very meaningful...
 # ... and we cannot clearly see variation in crime rates.
 # This is likely because the crime rate data is highly skewed.
 # I'll visualise LSOA crime rates in a histogram to confirm this.*

```
merged_crime24_drop['viol_rate'].hist(bins=50, figsize=(10,5))
plt.title('Histogram of Violent Crime Rates per LSOA (Past 24 Months)')
plt.xlabel('Violent Crimes per 1000 Residents')
plt.ylabel('Frequency')
plt.show()
```



In [30]: *# Normalising crime rates by logging them for visualisation purposes*
 logged = merged_crime24_drop.copy()
 logged['logged'] = np.log(logged['viol_rate'])

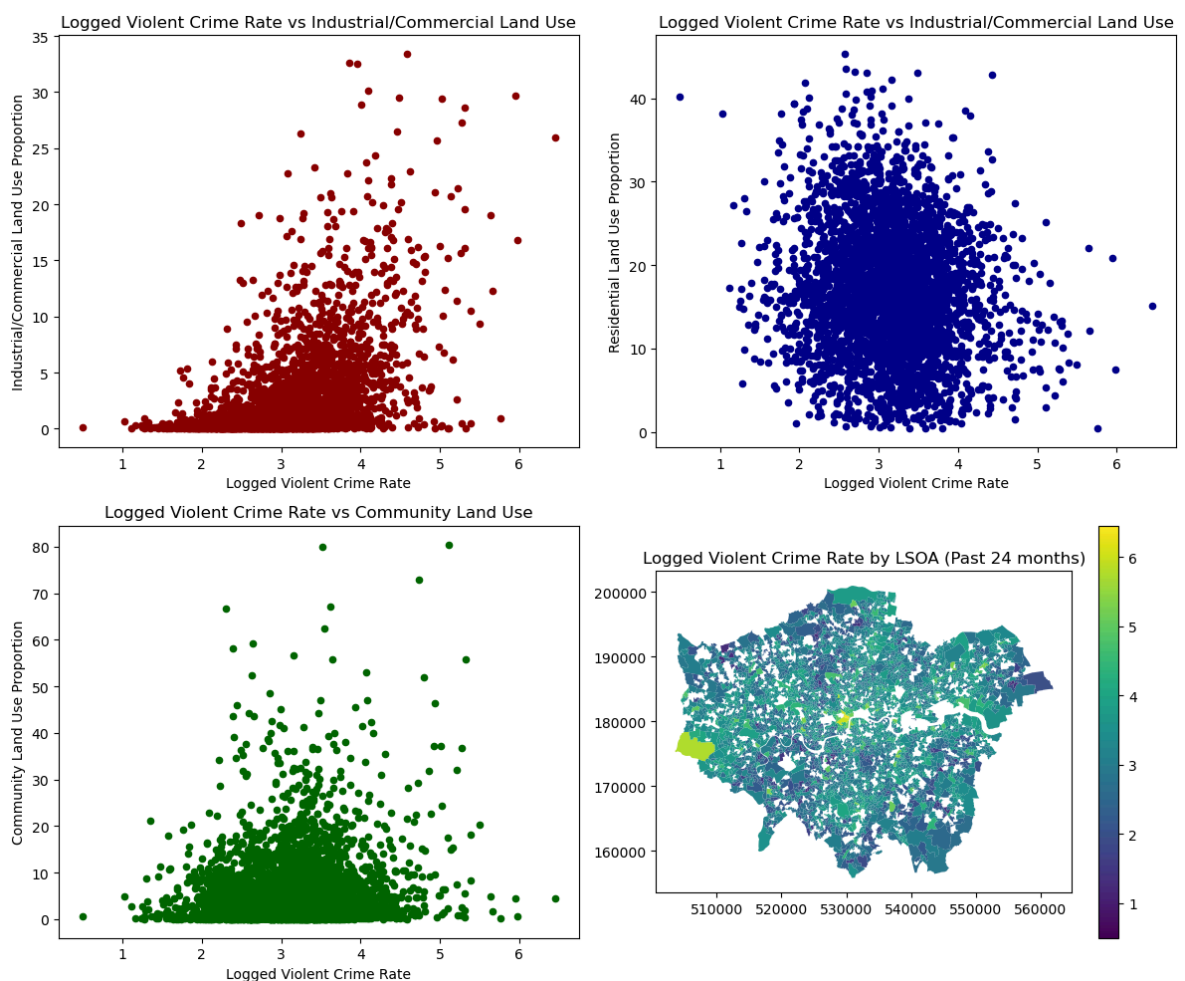
```
In [31]: # Plotting scatter plots of logged crime rates to visualise the relationship
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

logged.plot.scatter(x = 'logged', y = 'indcom_prop', ax = axes[0,0],
                    color = "darkred")
axes[0, 0].set_title('Logged Violent Crime Rate vs Industrial/Commercial Land Use Proportion')
axes[0, 0].set_xlabel('Logged Violent Crime Rate')
axes[0, 0].set_ylabel('Industrial/Commercial Land Use Proportion')

logged.plot.scatter(x = 'logged', y = 'res_prop', ax = axes[0,1],
                    color = "darkblue")
axes[0, 1].set_title('Logged Violent Crime Rate vs Industrial/Commercial Land Use Proportion')
axes[0, 1].set_xlabel('Logged Violent Crime Rate')
axes[0, 1].set_ylabel('Residential Land Use Proportion')

logged.plot.scatter(x = 'logged', y = 'commun_prop', ax = axes[1,0],
                    color = "darkgreen")
axes[1, 0].set_title('Logged Violent Crime Rate vs Community Land Use')
axes[1, 0].set_xlabel('Logged Violent Crime Rate')
axes[1, 0].set_ylabel('Community Land Use Proportion')

logged.plot(column='logged',
            legend=True,
            cmap='viridis', ax = axes [1,1])
axes[1, 1].set_title("Logged Violent Crime Rate by LSOA (Past 24 months)")
plt.tight_layout()
plt.show()
```



The scatter plots give us a general idea of the relationships between land use and violent crime in our data.

The next section conducts cluster analysis on violent crime and land use data. Data is first standardised and then clustered using *k*-Means and agglomerative methods. Outputs are visualised accordingly.

5.4 *k*-Means clustering

```
In [32]: # Standardising crime rates with RobustScaler
rs = RobustScaler(quantile_range=(10.0, 90.0))

normed = merged_crime24_drop[['LSOA11CD',
                              'res_prop', 'indcom_prop', 'commun_prop',
                              'viol_rate']].set_index('LSOA11CD')

# Applying RobustScaler
for c in normed.columns.values:
    normed[c] = rs.fit_transform(merged_crime24_drop[c].values.reshape(-1,1))
    print("The range of {} is [{} , {}]".format(c, normed[c].min(), normed[c].max()))
```

The range of res_prop is [-0.8214285714285716, 1.469387755102041].
The range of indcom_prop is [-0.140625, 5.078125].
The range of commun_prop is [-0.20289855072463767, 5.630434782608695].
The range of viol_rate is [-0.5873346223923877, 16.20961513295934].

```
In [33]: # Below, I'm comparing the silhouette scores of k-Means clustering using different k values

random_state_seed = 10

# Experimenting with 2-6 clusters
# Note: Code adapted from CASA0007 Practical 6
df_silhouette_score = pd.DataFrame({'n_cluster': [2,3,4,5,6],
                                    'silhouette_score': [0,0,0,0,0]})

for index, row in df_silhouette_score.iterrows():
    n_clusters = row['n_cluster']
    clusterer = sklc.KMeans(n_clusters=n_clusters,
                            random_state=random_state_seed).fit(normed)
    cluster_labels = clusterer.labels_
    # The silhouette_score gives the average value for all the samples.
    # This gives a perspective into the density and separation of the formed
    # clusters
    silhouette_avg = metrics.silhouette_score(normed,
                                              cluster_labels)
    print(f"For k = {n_clusters}, The average silhouette_score is {silhouette_avg}")
```

For k = 2, The average silhouette_score is 0.5660.
For k = 3, The average silhouette_score is 0.4462.
For k = 4, The average silhouette_score is 0.4319.
For k = 5, The average silhouette_score is 0.2349.
For k = 6, The average silhouette_score is 0.2426.

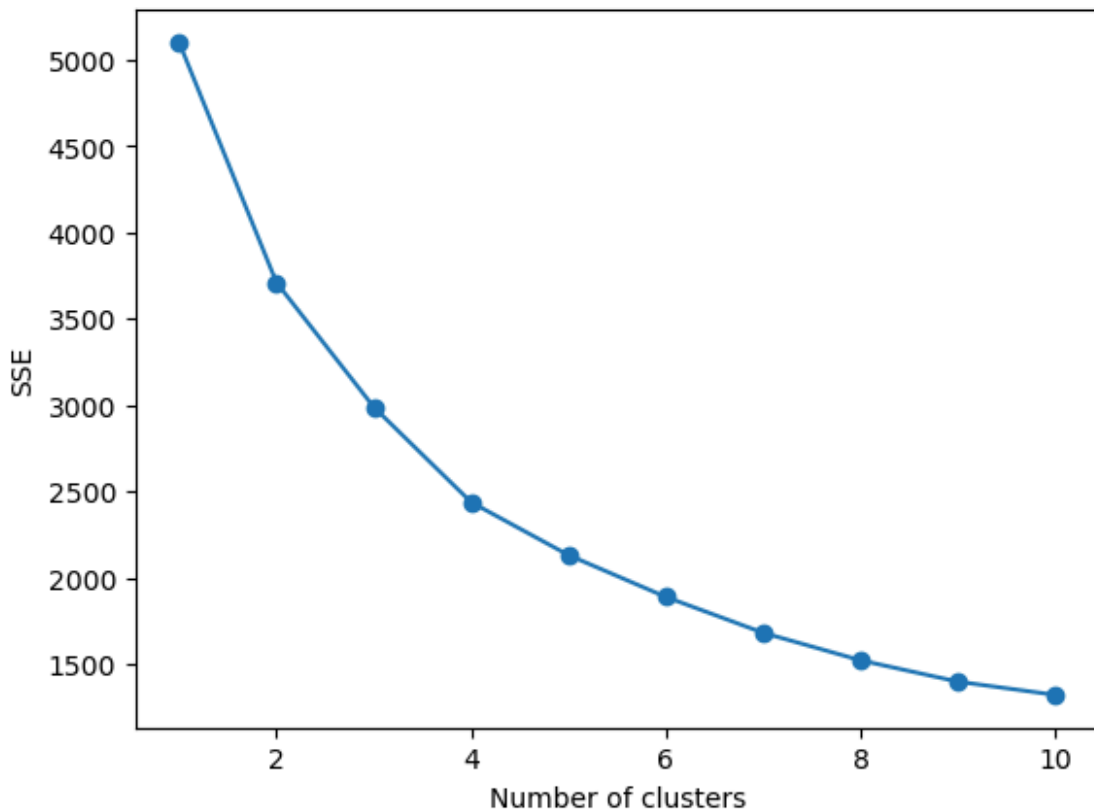
```
In [34]: # As a second check, I'm comparing the SSEs of clustering using different k values
# Calculating SSEs for different k values
list_SSE = []
min_k = 1
max_k = 10
range_k = range(min_k, max_k+1)
for i in range_k:
    km = KMeans(
        n_clusters=i, init='random',
        n_init=10, max_iter=300,
        tol=1e-04, random_state=0)
```

```

    )
    km.fit(normed)
    # inertia is a concept from physics. Roughly it means SSE of clustering.
    list_SSE.append(km.inertia_)

# plot
plt.plot(range_k, list_SSE, marker='o')
plt.xlabel('Number of clusters')
plt.ylabel('SSE')
plt.show()

```



Evaluating silhouette scores and SSEs (using the elbow method) for a range of k seem to suggest that $k=2$ might be a good cluster number. However, preliminary fitting and visualisation of k -Means using $k=2$ did not provide much insight: the low k meant that groups were large and masked LSOA heterogeneity within clusters. Therefore, while the above statistical metrics for k were considered, the following cluster analysis was also guided by interpretability considerations and judgment. It proceeds with k -means clustering using $k=3$.

```

In [35]: # Clustering with k=3
k_cluster = 3
random_seed = 123
kmeans = KMeans(n_clusters = k_cluster,
                 random_state = random_seed)
kmeans.fit(normed)

```

```

Out[35]:
KMeans
KMeans(n_clusters=3, random_state=123)

```

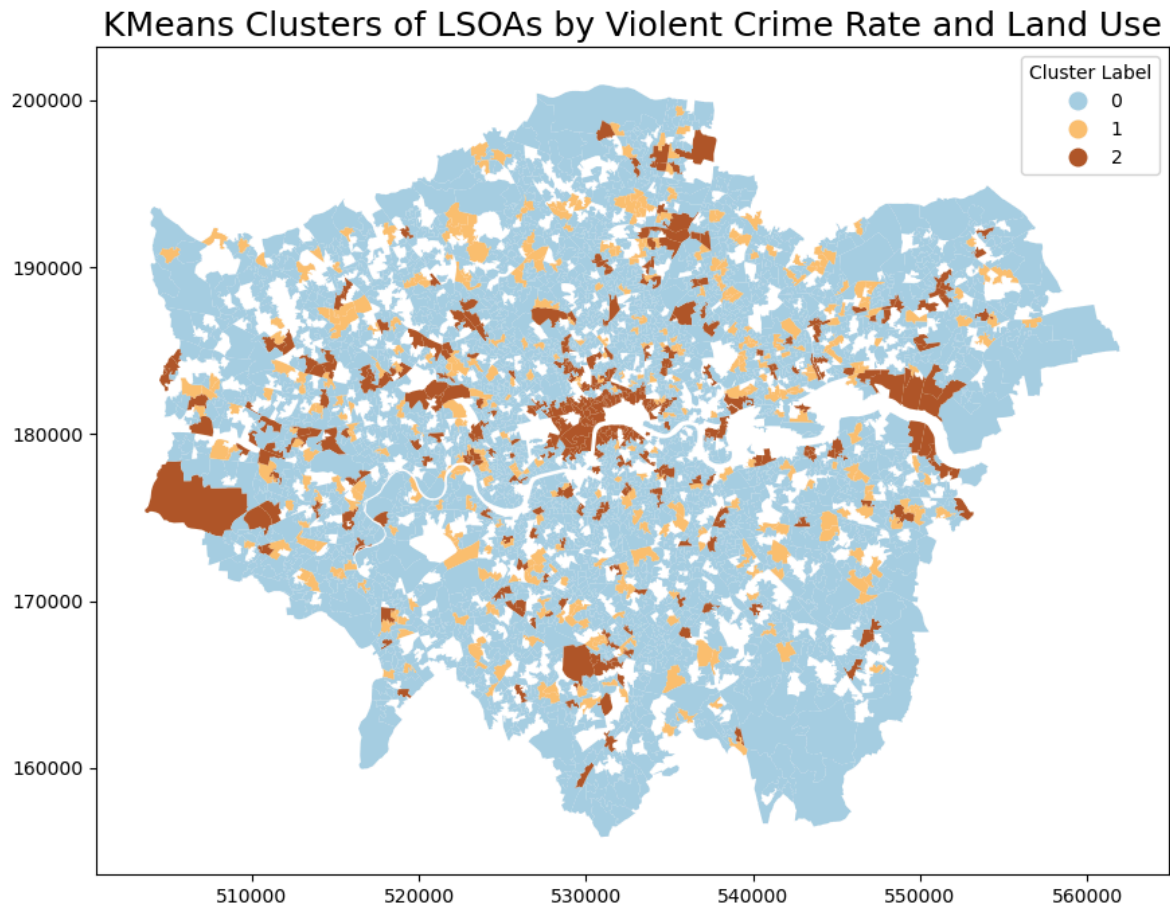
```

In [36]: # Extracting cluster labels from KMeans output and assigning them to geodata
merged_kmeans = merged_crime24_drop.copy()

```

```
merged_kmeans['cluster_nm'] = kmeans.labels_

ax = merged_kmeans.plot(column='cluster_nm',
                        categorical=True,
                        legend=True,
                        figsize=(12,8),
                        cmap='Paired')
ax.get_legend().set_title("Cluster Label")
ax.set_title("KMeans Clusters of LSOAs by Violent Crime Rate and Land Use",
```



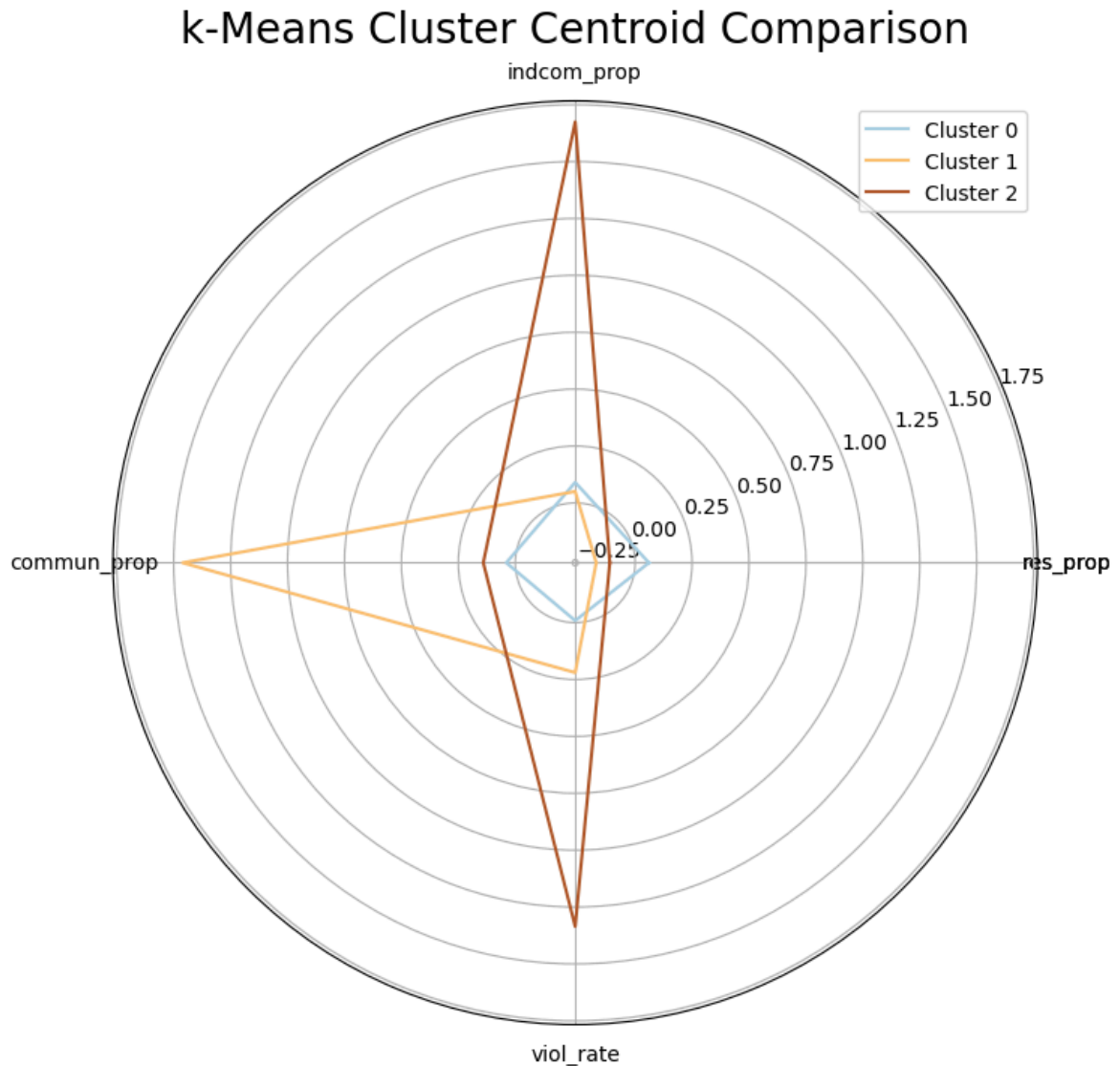
```
In [37]: # Function to plot radar plots
# Code adapted from CASA0006 Practical 7

def radar_plot_cluster_centroids(df_cluster_centroid, colours):
    categories = df_cluster_centroid.columns.values.tolist()
    categories = [*categories, categories[0]]
    label_loc = np.linspace(start=0, stop=2 * np.pi, num=len(categories))
    plt.figure(figsize=(12, 8))
    plt.subplot(polar=True)
    for index, row in df_cluster_centroid.iterrows():
        centroid = row.tolist()
        centroid = [*centroid, centroid[0]]
        label = f"Cluster {index}"
        color = colours[index]
        plt.plot(label_loc, centroid, label=label, color = color)
    lines, labels = plt.thetagrids(np.degrees(label_loc), labels=categories)
    plt.legend()
```

```
In [38]: # Visualising k-Means output with a radar plot
kmeans_centroids = pd.DataFrame(kmeans.cluster_centers_,
                                columns = normed.columns)

# Matching colours
colours = plt.colormaps['Paired'](np.linspace(0, 1, k_cluster))
```

```
radar_plot_cluster_centroids(kmeans_centroids, colours)
plt.title('k-Means Cluster Centroid Comparison', size=20, y=1.05);
```



The *k*-Means clustering output suggests that the cluster exhibiting the highest violent crime rate (**Cluster 2, brown**) also has the highest industrial/commercial land use proportion. Many of the LSOAs can be seen in highly commercial and less residential parts of central London. The cluster with the lowest crime rate (**Cluster 0, blue**) has a lower industrial/commercial presence and the highest residential land use proportion, suggesting a protective effect of residential presence on crime. Low industrial/commercial presence coupled with greater community-service land use appears to be associated with less violent crime (**Cluster 1, yellow**).

5.5 Agglomerative clustering

```
In [39]: def plot_dendrogram(model, **kwargs):
# Creating the counts of samples under each node
counts = np.zeros(model.children_.shape[0])
n_samples = len(model.labels_)
for i, merge in enumerate(model.children_):
    current_count = 0
    for child_idx in merge:
        if child_idx < n_samples:
```

```

        current_count += 1 # leaf node
    else:
        current_count += counts[child_idx - n_samples]
    counts[i] = current_count

# Creating linkage matrix and then plot the dendrogram
linkage_matrix = np.column_stack(
    [model.children_, model.distances_, counts]
).astype(float)

# Plotting dendrogram
dendrogram(linkage_matrix, leaf_rotation=90., **kwargs)

```

In [40]: # First setting distance_threshold = 0 and n_clusters = None for dendrogram

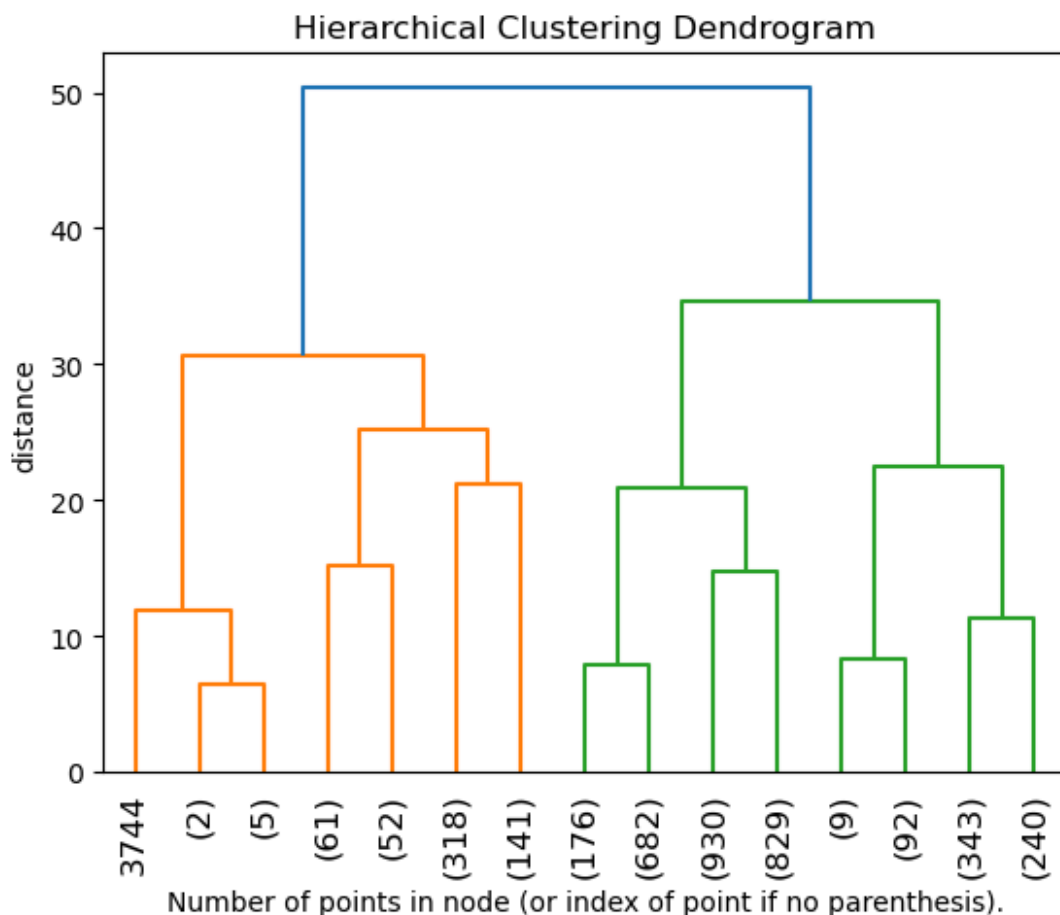
```

agg_cluster = AgglomerativeClustering(distance_threshold=0,
                                       n_clusters=None).fit(normed)

ax = plt.gca()
plt.title("Hierarchical Clustering Dendrogram")

# plot the top three levels of the dendrogram
plot_dendrogram(agg_cluster, truncate_mode="level", p=3)
plt.xlabel("Number of points in node (or index of point if no parenthesis).")
plt.ylabel('distance');

```



In [41]: # Similarly, calculating silhouette scores for a range of cluster numbers

```

# Note: Code adapted from CASA0007 Practical 6
df_silhouette_score = pd.DataFrame({'n_cluster': [2,3,4,5,6],
                                    'silhouette_score': [0,0,0,0,0]})

for index, row in df_silhouette_score.iterrows():
    n_clusters = row['n_cluster']
    clusterer = sklc.AgglomerativeClustering(n_clusters=n_clusters,

```

```

        distance_threshold=None).fit(normed)
    cluster_labels = clusterer.labels_
    silhouette_avg = metrics.silhouette_score(normed,
                                              cluster_labels)
    print(f"For n_clusters = {n_clusters}, The average silhouette_score is {

```

For n_clusters = 2, The average silhouette_score is 0.4740.
 For n_clusters = 3, The average silhouette_score is 0.3102.
 For n_clusters = 4, The average silhouette_score is 0.3164.
 For n_clusters = 5, The average silhouette_score is 0.3215.
 For n_clusters = 6, The average silhouette_score is 0.2662.

Again, balancing statistical metrics and interpretability, I'll proceed to conduct agglomerative clustering with 5 clusters.

```

In [42]: # Now setting n_clusters = 5
agg_cluster = AgglomerativeClustering(distance_threshold=None,
                                       n_clusters=5).fit(normed)
agg_cluster

```

```

Out[42]: AgglomerativeClustering
AgglomerativeClustering(n_clusters=5)

```

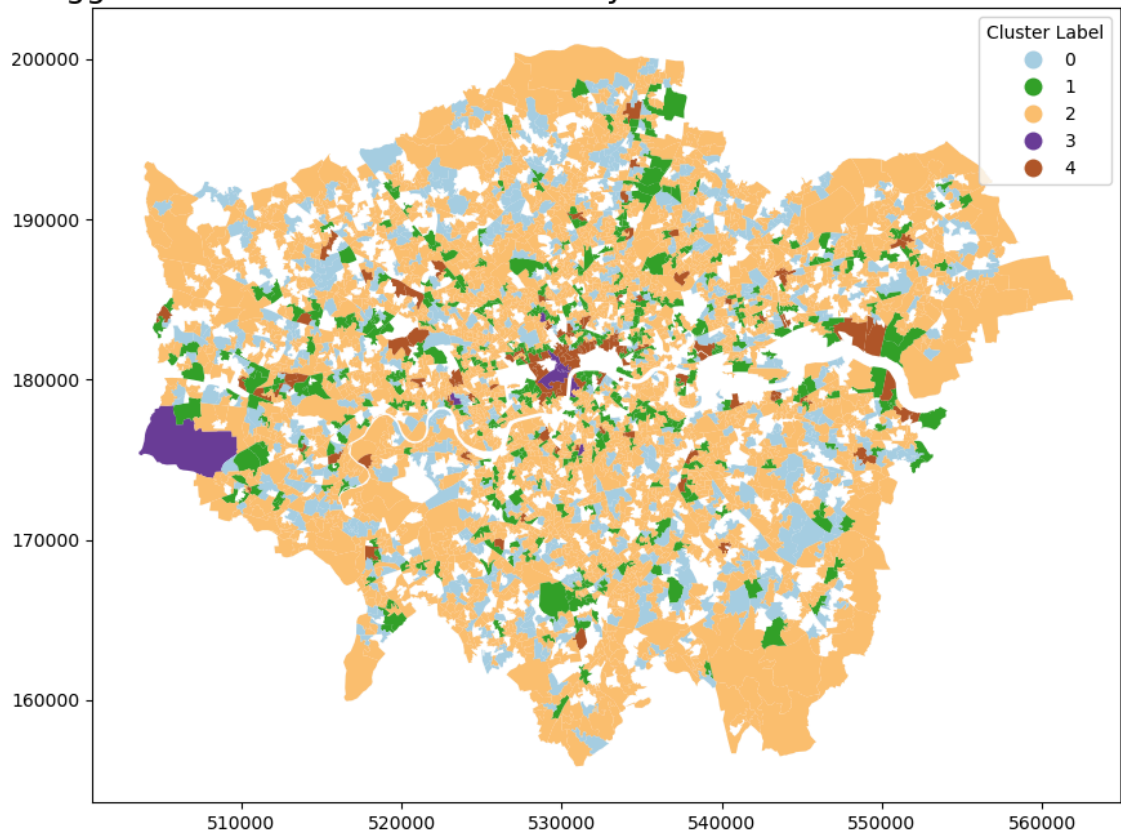
```

In [43]: # Extracting cluster labels from agglomerative clustering output and assign
merged_agg = merged_crime24_drop.copy()
merged_agg['cluster_nm'] = agg_cluster.labels_

# Plotting
ax = merged_agg.plot(column='cluster_nm',
                    categorical=True,
                    legend=True,
                    figsize=(12,8),
                    cmap='Paired')
ax.get_legend().set_title("Cluster Label")
ax.set_title("Agglomerative Clusters of LSOAs by Violent Crime Rate and Land

```

Agglomerative Clusters of LSOAs by Violent Crime Rate and Land Use



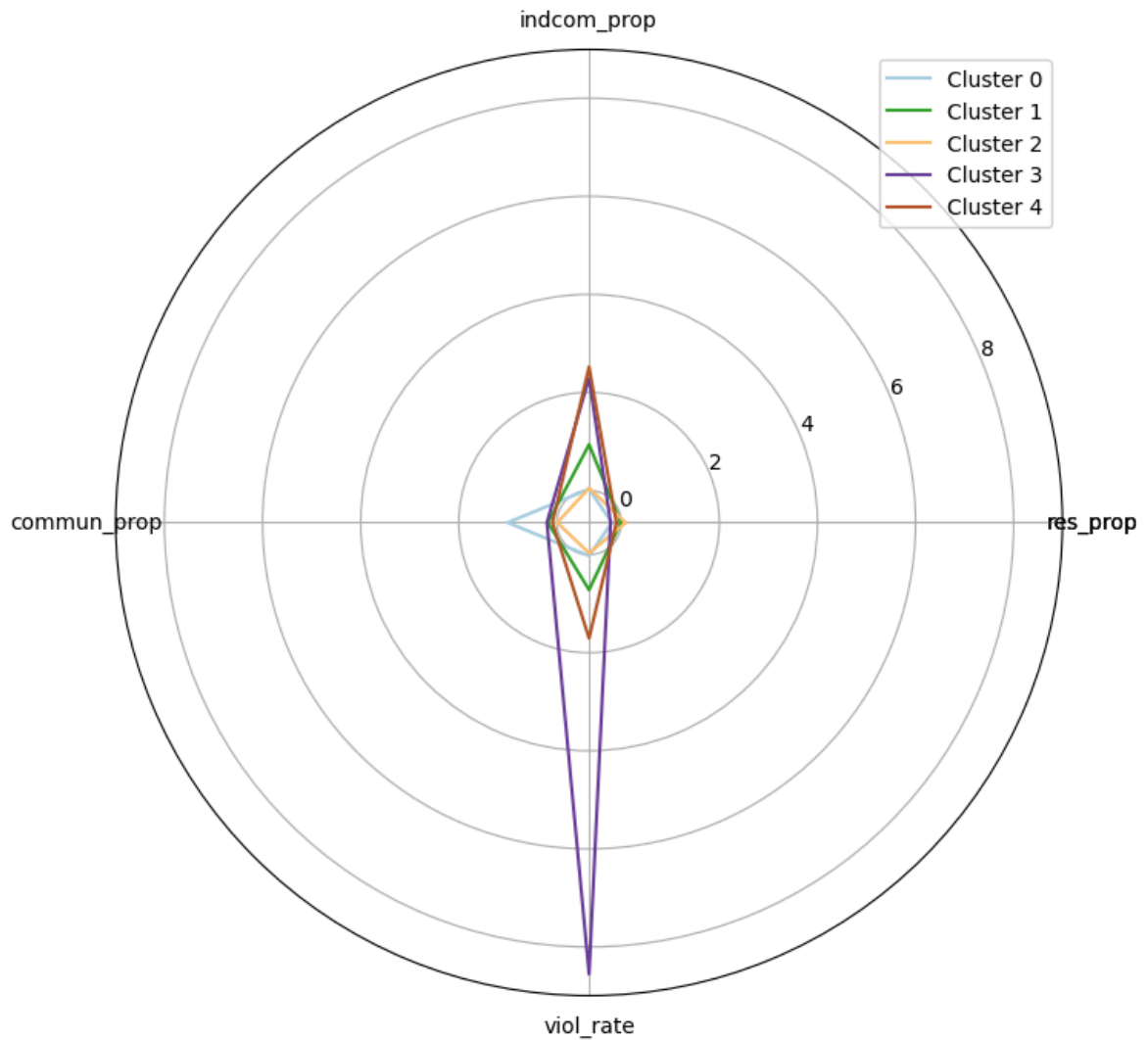
```
In [44]: df_agg = normed.copy()
df_agg['cluster_nm'] = agg_cluster.labels_

# Calculating centroids
agg_centroids = df_agg.groupby('cluster_nm').mean()

# Matching colours
agg_ncluster = len(np.unique(agg_cluster.labels_))
agg_colours = plt.colormaps['Paired'](np.linspace(0, 1, agg_ncluster))

# Plotting
radar_plot_cluster_centroids(agg_centroids, agg_colours)
plt.title('Agglomerative Clusters Centroid Comparison', size=20, y=1.05);
```

Agglomerative Clusters Centroid Comparison



Similar to the *k*-Means output, the cluster with the highest violent crime rate (**Cluster 3, purple**) has among the highest industrial/commercial land use proportion, while the lowest violent crime rate (**Cluster 2, yellow**) has the highest residential land use proportion. The cluster with the most community-related land use (**Cluster 0, blue**) also has among the lowest violent crime rates.

5.6 Difference-in-differences Analysis with Historical data

This section conducts DiD analysis on historical crime data to assess CDY's causal influence on local violent crime.

Identifying treated and control group

The following steps are taken:

- Aggregate crimes for all months available
- Calculate monthly average crime rate using demographic data
- Identify treated LSOA using CDY POI information

- Calculate correlation coefficient for pre-treatment months to identify LSOA with pre-treatment trend similar to treated LSOA

```
In [45]: # Summing crime counts for each LSOA per YYYYMM

# Identifying YYYYMM columns
monthly_cols = [col for col in crimehistdf.columns if col.isdigit()]

# Rearranging YYYYMM columns so that they are chronological
monthly_cols = sorted(monthly_cols)

# Summing across categories
monthly_by_lsoa = crimehistdf.groupby(['LSOA Code', 'LSOA Name'])[monthly_cols].sum()

monthly_by_lsoa.head()
```

```
Out[45]:
```

	LSOA Code	LSOA Name	201004	201005	201006	201007	201008	201009	201010
0	E01000006	Barking and Dagenham 016A	3	2	0	4	2	2	
1	E01000007	Barking and Dagenham 015A	14	8	14	3	8	3	
2	E01000008	Barking and Dagenham 015B	9	4	5	2	4	1	
3	E01000009	Barking and Dagenham 016B	3	6	7	1	7	7	
4	E01000011	Barking and Dagenham 016C	5	2	0	6	1	2	

5 rows x 156 columns

```
In [46]: # Merging monthly crime data to demographic data to calculate crime rates
merged_hist = pd.merge(gdf, monthly_by_lsoa,
                        left_on='LSOA11CD', right_on='LSOA Code',
                        how='inner')

# Keeping relevant columns
merged_hist = merged_hist.drop(columns = ['MSOA11CD', 'MSOA11NM', 'LAD11CD',

# Calculating monthly rate per 1000 residents
merged_hist_rate = merged_hist.copy()
for col in merged_hist_rate.columns:
    if col.isdigit():
        merged_hist_rate[col] = (merged_hist_rate[col]/merged_hist_rate['USL
merged_hist_rate.head()
```

Out [46]:

	LSOA11NM	USUALRES	geometry	201004	201005	201006
LSOA11CD						
E01000006	Barking and Dagenham 016A	1703	POLYGON ((545122.049 184314.931, 545271.917 18...	1.761597	1.174398	0.000000
E01000007	Barking and Dagenham 015A	1391	POLYGON ((544180.347 184700.582, 544317.196 18...	10.064702	5.751258	10.064702
E01000008	Barking and Dagenham 015B	1544	MULTIPOLYGON (((543741.001 184566, 543806.15 1...	5.829016	2.590674	3.238342
E01000009	Barking and Dagenham 016B	1773	POLYGON ((544499.837 184704.366, 544630.292 18...	1.692047	3.384095	3.948111
E01000011	Barking and Dagenham 016C	1634	POLYGON ((544523.402 184789.039, 544667.055 18...	3.059976	1.223990	0.000000

5 rows × 157 columns

In [47]:

```
# Taking POI information from Wikipedia
poi = "Coal Drop's Yard"
poi_lon = -0.126444
poi_lat = 51.536336

# Identifying the LSOA containing Coal Drops Yard

# 1. Creating point geometry from POI
poi_point = Point(poi_lon, poi_lat)

# 2. Converting POI to gdf and aligning CRS
poi_gdf = gpd.GeoDataFrame([{'Name': poi}],
                             geometry=[poi_point],
                             crs = 'EPSG:4326')
poi_gdf = poi_gdf.to_crs(gdf.crs)

# 3. Joining poi_gdf to LSOA gdf
poi_lsoa = gpd.sjoin(poi_gdf, gdf, how = 'left', predicate = 'within')

# 4. Identifying LSOA
print(poi_lsoa[['Name', 'LSOA11CD', 'LSOA11NM']])
```

	Name	LSOA11CD	LSOA11NM
0	Coal Drop's Yard	E01000953	Camden 022B

In [48]:

```
# Crime data is missing for Camden 022B, so I'm identifying LSOAs closest to

# Extracting row and centroid of Camden 022B LSOA
cdy_lsoa = gdf.loc[gdf['LSOA11NM'] == 'Camden 022B', 'geometry']
cdy_centroid = cdy_lsoa.geometry.centroid.values[0]
```

```

# Identifying LSOAs that have crime data AND are closest to Camden 022B
# Calculating distance between Camden 022B's centroid and centroids of LSOAs
merged_hist_rate = merged_hist_rate.copy()
merged_hist_rate['distance'] = merged_hist_rate.geometry.centroid.distance(c

# Identifying nearest LSOAs
nearest_lsoas = merged_hist_rate[['LSOA11NM', 'distance']].sort_values('dist
print(f"3 closest LSOAs to Coal Drop's Yard:")
print(f"{nearest_lsoas.head(5)} \n")

print(f"As proxy for {poi_lsoa.iloc[0]['LSOA11NM']}, {nearest_lsoas.iloc[0]}

```

```

3 closest LSOAs to Coal Drop's Yard:
      LSOA11NM      distance
LSOA11CD
E01000958    Camden 019D  488.068032
E01002709  Islington 019C  545.027296
E01002711  Islington 021B  546.079085
E01000952    Camden 022A  550.224508
E01002710  Islington 019D  609.525519

```

As proxy for Camden 022B, Camden 019D will be used as the treatment group in our DiD analysis of Coal Drop's Yard's impact on violent crime rates.

In [49]: # Separating pre- and post-treatment periods: 201004–201810 and 201811–202303

```

pre = ['201004', '201005', '201006', '201007', '201008', '201009', '201010',
       '201101', '201102', '201103', '201104', '201105', '201106', '201107',
       '201110', '201111', '201112', '201201', '201202', '201203', '201204',
       '201207', '201208', '201209', '201210', '201211', '201212', '201301',
       '201304', '201305', '201306', '201307', '201308', '201309', '201310',
       '201401', '201402', '201403', '201404', '201405', '201406', '201407',
       '201410', '201411', '201412', '201501', '201502', '201503', '201504',
       '201507', '201508', '201509', '201510', '201511', '201512', '201601',
       '201604', '201605', '201606', '201607', '201608', '201609', '201610',
       '201701', '201702', '201703', '201704', '201705', '201706', '201707',
       '201710', '201711', '201712', '201801', '201802', '201803', '201804',
       '201807', '201808', '201809', '201810']
print(f"No. of months pre-treatment: {len(pre)}")

post = ['201811', '201812', '201901', '201902', '201903', '201904', '201905',
        '201908', '201909', '201910', '201911', '201912', '202001', '202002',
        '202005', '202006', '202007', '202008', '202009', '202010', '202011',
        '202102', '202103', '202104', '202105', '202106', '202107', '202108',
        '202111', '202112', '202201', '202202', '202203', '202204', '202205',
        '202208', '202209', '202210', '202211', '202212', '202301']
print(f"No. of months post-treatment: {len(post)}")

```

```

No. of months pre-treatment: 103
No. of months post-treatment: 51

```

In [50]: # Subsetting to pre-treatment columns for (i) treatment group and (ii) every

```

pre_treated = merged_hist_rate.loc[merged_hist_rate['LSOA11NM'] == 'Camden 019D']
pre_controls = merged_hist_rate[pre]

# Computing correlation with all other LSOAs
corrs = pre_controls.apply(lambda x: x.corr(pre_treated.iloc[0]), axis=1)

# Returning top matches
print(corrs.sort_values(ascending=False).head())

```

```
print(f"A possible LSOA demonstrating a similar pre-treatment crime rate tre
```

LSOA11CD

E01000958 1.000000

E01033083 0.604583

E01004570 0.584119

E01004626 0.578186

E01002042 0.551810

dtype: float64

A possible LSOA demonstrating a similar pre-treatment crime rate trend is Ho
unslow 018F.

```
In [51]: # Now that treated and control groups have been identified, only years for v  
# Getting yearly average rates
```

```
# Identifying YYYYMM columns
```

```
monthly_cols = [col for col in merged_hist.columns if col.isdigit()]
```

```
# Summing columns that start with the same year
```

```
yearly_agg = pd.DataFrame()
```

```
for year in sorted(set(col[:4] for col in monthly_cols)):
```

```
    year_cols = [col for col in monthly_cols if col.startswith(year)]
```

```
    # Calculating year totals
```

```
    yearly_agg[year] = merged_hist[year_cols].sum(axis=1)
```

```
# Adding LSOA11NM to yearly_agg
```

```
yearly_agg.insert(0, 'LSOA11NM', merged_hist['LSOA11NM'])
```

```
yearly_agg['USUALRES'] = merged_hist['USUALRES']
```

```
# Dropping 2010 and 2023 because they are incomplete years
```

```
yearly_agg = yearly_agg.drop(columns = ["2010", "2023"])
```

```
# Calculating rate from aggregates
```

```
yearly_rate = yearly_agg.copy()
```

```
for col in yearly_rate.columns:
```

```
    if col.isdigit():
```

```
        yearly_rate[col] = (yearly_rate[col]/yearly_rate['USUALRES'])*1000
```

```
yearly_rate.head()
```

```
print("Yearly crime rates:")
```

```
yearly_rate.head()
```

Yearly crime rates:

Out [51]:	LSOA11NM	2011	2012	2013	2014	2015
LSOA11CD						
E01000006	Barking and Dagenham 016A	7.633588	12.331180	13.505578	10.569583	19.377569
E01000007	Barking and Dagenham 015A	69.015097	57.512581	45.291157	61.107117	115.025162
E01000008	Barking and Dagenham 015B	18.134715	19.430052	22.668394	29.145078	31.088083
E01000009	Barking and Dagenham 016B	43.429216	27.072758	34.968979	41.737169	39.481105
E01000011	Barking and Dagenham 016C	15.911873	20.195838	12.851897	26.315789	24.479804

```
In [52]: # LSOAs to keep for DiD analysis
did_lsoas = ['E01000958', 'E01033083'] # Camden 019D and Hounslow 018F

yearly_long = yearly_rate.reset_index().melt(
    id_vars = ['LSOA11CD', 'LSOA11NM'],
    value_vars = [col for col in yearly_rate.columns if col.isdigit()],
    var_name = 'year',
    value_name = 'crimrate')

yearly_long = yearly_long[yearly_long['LSOA11CD'].isin(did_lsoas)]

yearly_long.head()
```

Out [52]:	LSOA11CD	LSOA11NM	year	crimrate
910	E01000958	Camden 019D	2011	25.176678
4566	E01033083	Hounslow 018F	2011	61.899038
5563	E01000958	Camden 019D	2012	32.243816
9219	E01033083	Hounslow 018F	2012	63.100962
10216	E01000958	Camden 019D	2013	24.293286

```
In [53]: # Plotting treated and control group crime rates for pre-treatment period
yearly_long['year'] = pd.to_datetime(yearly_long['year'], format='%Y')

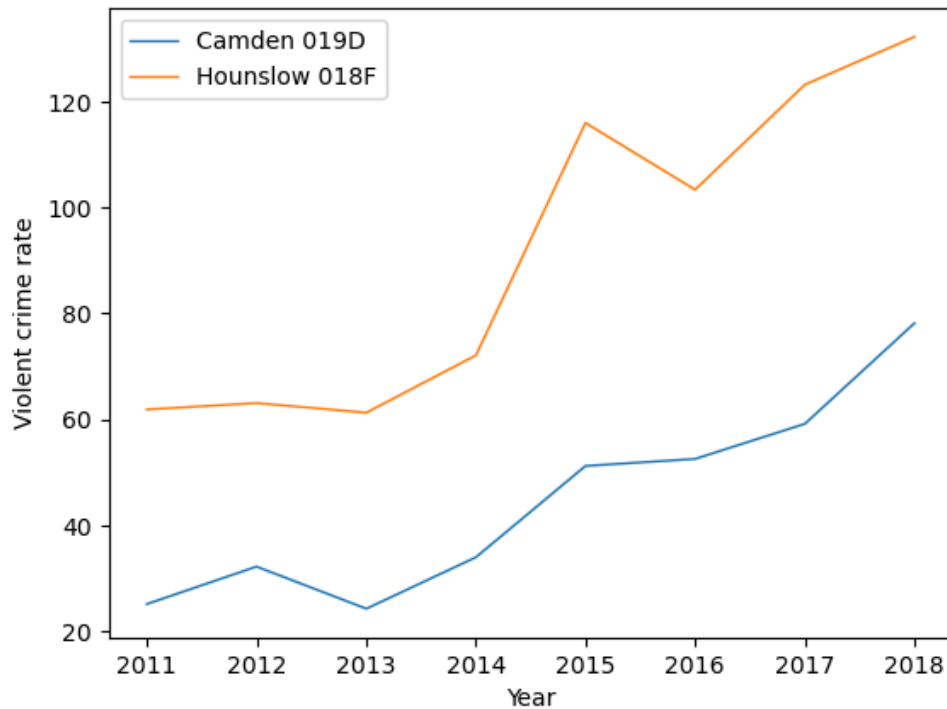
sns.lineplot(data = yearly_long[
    (yearly_long['year'].dt.year >= 2011) &
    (yearly_long['year'].dt.year <= 2018)],
    x='year',
    y='crimrate',
```

```

        hue='LSOA11NM',
        linewidth = 1)
plt.title('Pre-treatment Average Violent Crime Rates in Camden 019D and Hounslow 018F')
plt.xlabel('Year')
plt.ylabel('Violent crime rate')
plt.legend();

```

Pre-treatment Average Violent Crime Rates in Camden 019D and Hounslow 018F



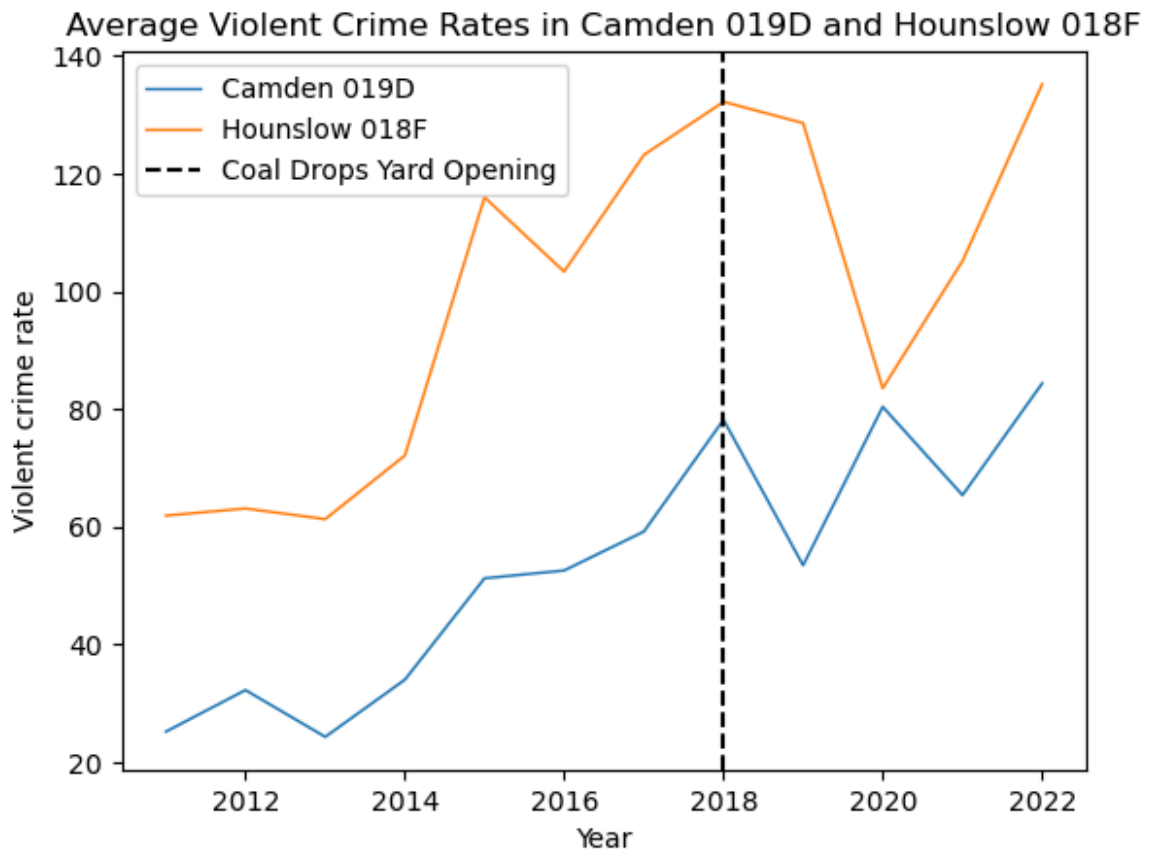
Apart from an uptick in 2015 for Hounslow 018F, the two LSOAs demonstrate largely parallel trends in pre-treatment crime rates.

Visualising post-treatment effects

```

In [54]: sns.lineplot(data = yearly_long,
                      x='year',
                      y='crimrate',
                      hue='LSOA11NM',
                      linewidth = 1)
# Adding a line for 2018 to represent the opening of Coal Drops Yard
plt.axvline(pd.to_datetime('2018'),color='black',linestyle='dashed', label='2018')
plt.title('Average Violent Crime Rates in Camden 019D and Hounslow 018F')
plt.xlabel('Year')
plt.ylabel('Violent crime rate')
plt.legend();

```



The plot comparing the behaviour of the treated and control groups indicates a relatively sharp decline in average crime rate in 2019, the year after CDY opened. In contrast, Hounslow 018F exhibited a gentler decrease. The following step evaluates the statistical significance of this.

Evaluating statistical significance of treatment effect

```
In [55]: # Classifying post/treatment/postXtreatment rows for OLS model

# 1. Creating variable that is 1 if after Coal Drop's Yard opening (starting
yearly_long['post'] = np.where(yearly_long['year'] >= '2019',1,0)

# 2. Creating variable that is 1 if Camden 019D (treatment), and 0 if Hounslow 018F
yearly_long['treatment'] = np.where(yearly_long['LSOA11CD'] == "E01000958",

# 3. Creating variable that is 1 if Camden 019D and after Coal Drop's Yard opening
yearly_long['post_treatment'] = yearly_long['post']*yearly_long['treatment']

In [56]: # Including land use columns as additional independent variables to control
yearly_long = pd.merge(yearly_long, landuse[['LSOA code', 'res_prop', 'indcc
left_on='LSOA11CD', right_on='LSOA code',
how='left']).drop(columns = 'LSOA code')

yearly_long.head()
```

Out [56]:	LSOA11CD	LSOA11NM	year	crimerate	post	treatment	post_treatment	res_pi
0	E01000958	Camden 019D	2011- 01-01	25.176678	0	1	0	1
1	E01033083	Hounslow 018F	2011- 01-01	61.899038	0	0	0	1
2	E01000958	Camden 019D	2012- 01-01	32.243816	0	1	0	1
3	E01033083	Hounslow 018F	2012- 01-01	63.100962	0	0	0	1
4	E01000958	Camden 019D	2013- 01-01	24.293286	0	1	0	1

```
In [57]: did_model_year = ols('crimerate ~ res_prop + indcom_prop + commun_prop + pos
print(did_model_year.summary())
```

OLS Regression Results

```

=====
==
Dep. Variable:          crimerate    R-squared:                0.5
81
Model:                  OLS          Adj. R-squared:            0.5
18
Method:                 Least Squares    F-statistic:              9.2
45
Date:                   Sun, 20 Apr 2025    Prob (F-statistic):       0.0004
85
Time:                   22:44:15          Log-Likelihood:           -107.
75
No. Observations:       24              AIC:                      22
3.5
Df Residuals:           20              BIC:                      22
8.2
Df Model:               3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025
0.975]					

Intercept	0.0714	0.022	3.289	0.004	0.026
0.117					
res_prop	1.3894	0.167	8.337	0.000	1.042
1.737					
indcom_prop	2.2222	0.289	7.689	0.000	1.619
2.825					
commun_prop	-0.0175	0.625	-0.028	0.978	-1.321
1.286					
post	21.4844	14.458	1.486	0.153	-8.674
51.643					
treatment	-0.0071	0.048	-0.147	0.885	-0.107
0.093					
post_treatment	4.7965	20.447	0.235	0.817	-37.854
47.448					

```

=====
==
Omnibus:                3.778    Durbin-Watson:            0.9
03
Prob(Omnibus):           0.151    Jarque-Bera (JB):         1.5
13
Skew:                    0.158    Prob(JB):                 0.4
69
Kurtosis:                1.811    Cond. No.                  1.72e+
21
=====

```

```

==
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 6.41e-39. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```

Interpreting output

- **Residential land use** and **industrial/commercial land use** are positively related to violent crime and statistically significant at the 0.05 level, while **community-related land use** has a statistically insignificant negative relationship to violent crime.
- Comparing **overall pre- and post-treatment**, there was a general but statistically insignificant increase in crime for both LSOAs – this could be due to random chance.
- Comparing **treated and control LSOAs**, the difference in crime over the entire period is statistically insignificant.
- The **DiD estimator** (`post_treatment`) is positive but statistically insignificant, suggesting that CDY led to an increase in post-treatment crime in Camden 019D relative to Hounslow 018F, but that this could be due to random chance.

6. Results and discussion

[\[Back to top \]](#)

6.1 Implications of land use on violent crime

RQ1: What clusters and distributions can be observed from violent crime rates and land use characteristics across LSOAs?

Both *k*-Means and agglomerative clustering results suggest that violent crime may be correlated with industrial/commercial presence in LSOAs, and decreases with residential presence, consistent with literature (e.g. Wo & Kim, 2022). Community-related land use demonstrates a mediating impact on violent crime. Mapping clusters suggest a concentration of high-industrial/commercial land use, high-violent crime LSOAs in central London, which is unsurprising.

6.2 Impact of Coal Drops Yard

RQ2: Did the Coal Drops Yard development in King's Cross have causal impacts on local violent crime?

While comparisons of violent crime trends in treated and control LSOAs visually indicate a decrease in crime the year following CDY opening, the DiD estimator instead suggests an increase, though statistically insignificant, in violent crime for the treated LSOA post-2018.

The lack of a conclusive causal inference aligns with Klinenberg's (2018) writing regarding the trickiness of determining gentrification's impacts on local crime. Indeed, with its upscale boutiques and restaurants, CDY can be considered a gentrifying force in King's Cross. Whether it attracted more crime by increasing opportunities or protected against crime through strengthening the "eyes on the street" phenomenon is difficult to establish within this study's scope.

7. Conclusion

Overall, this study highlights interesting relationships between land use patterns and violent crime, with far-reaching implications for urban land use policy. The protective effects of residential and community-related land use and the exacerbating effects of industrial/commercial land use appear to align more with views that mixed land use heightens propensity for crime. Nevertheless, the inconclusive results of CDY's case study suggest scope for further, more nuanced work.

7.1 Study limitations

Nelson et al. (2001) note that researching violent crime is constrained by data quality; it is important to note that official data offers only a partial representation of reality (e.g. sexual offences are unavailable due to data protection laws). Analyses of violent crime would benefit from a wider range of crime data and/or qualitative input.

8. References

[\[Back to top \]](#)

1. Bouguettaya, A., Yu, Q., Liu, X., Zhou, X. and Song, A. (2015). 'Efficient agglomerative hierarchical clustering'. *Expert Systems with Applications*, 42 (5), pp. 2785–2797. doi: 10.1016/j.eswa.2014.09.054.
2. Browning, C. R., Byron, R. A., Calder, C. A., Krivo, L. J., Kwan, M.-P., Lee, J.-Y. and Peterson, R. D. (2010). 'Commercial Density, Residential Concentration, and Crime: Land Use Patterns and Violence in Neighborhood Context'. *Journal of Research in Crime and Delinquency*, 47 (3), pp. 329–357. doi: 10.1177/0022427810365906.
3. Haylock, S., Boshari, T., Alexander, E. C., Kumar, A., Manikam, L. and Pinder, R. (2020). 'Risk factors associated with knife-crime in United Kingdom among young people aged 10–24 years: a systematic review'. *BMC Public Health*, 20 (1), p. 1451. doi: 10.1186/s12889-020-09498-4.
4. Jacobs, J. (1961). *The Death and Life of Great American Cities*. New York, NY: Random House.
5. Klinenberg, E. (2018). *Palaces for the people: how to build a more equal and united society*. London: The Bodley Head.
6. Nelson, A. L., Bromley, R. D. F. and Thomas, C. J. (2001). 'Identifying micro-spatial and temporal patterns of violent crime and disorder in the British city centre'. *Applied Geography*, 21 (3), pp. 249–274. doi: 10.1016/S0143-6228(01)00008-X.
7. Stucky, T. D., & Ottensmann, J. R. (2009). Land use and violent crime. *Criminology*, 47(4), 1223-1264.
8. Taylor, R. B. (1988). *Human territorial functioning: an empirical, evolutionary perspective on individual and small group territorial cognitions, behaviors, and consequences*. Cambridge: Cambridge University Press (Environment and behavior series).

9. Thakker, Z. L. and H. Buch, S. (2024). 'Effect of Feature Scaling Pre-processing Techniques on Machine Learning Algorithms to Predict Particulate Matter Concentration for Gandhinagar, Gujarat, India'. *International Journal of Scientific Research in Science and Technology*, pp. 410–419. doi: 10.32628/IJSRST52411150.
10. Vanderplas, J. T. (2022). *Python data science handbook: essential tools for working with data*. Second edition. Sebastopol, CA: O'Reilly Media.
11. Wo, J. C. and Kim, Y.-A. (2022). 'Neighborhood Effects on Crime in San Francisco: An Examination of Residential, Nonresidential, and "Mixed" Land Uses'. *Deviant Behavior*, 43 (1), pp. 61–78. doi: 10.1080/01639625.2020.1779988.